Projet : gestion des livraisons et chargement d'un camion

Le code est dans le dossier `src/`. Le fichier `README.md` explique comment le lancer.

1. Langage choisi

J'ai fait le projet en **Python**. Le code se lit presque comme de l'algorithmique sur papier, ce qui permet de se concentrer sur les algorithmes. Python a aussi un dictionnaire intégré, qui est une table de hachage : c'est exactement ce qu'il faut pour retrouver un colis par son identifiant.

Python a deux défauts : il est plus lent que le C et il consomme plus de mémoire. Je les ai mesurés (partie 4) et j'en ai tenu compte dans certains choix. Je n'ai utilisé aucune bibliothèque externe. Les tris, le calcul du chargement et la règle de priorité sont codés à la main.

Le code est découpé en plusieurs fichiers :

- `colis.py` : la classe `Colis` (id, poids, valeur, priorité, destination) ;
- `entrepot.py` : la collection de colis (ajout, recherche, tris, statistiques) ;
- `tri.py` : le tri fusion et le tri par dénombrement ;
- `chargement.py` : le choix des colis à mettre dans le camion ;
- `main.py` : le menu ;
- `tests.py` : les tests ;
- `benchmark.py` : les mesures de temps.

2. Les choix algorithmiques

Choix 1 : ranger les colis dans un dictionnaire (table de hachage)

Problème	Il faut pouvoir ajouter des colis et en retrouver un très vite à partir de son identifiant, même avec 100 000 colis.
Solution	Je range les colis dans un dictionnaire `{identifiant: colis}`. Le dictionnaire calcule un « hash » de l'identifiant qui indique directement où est rangé le colis, donc il n'a pas besoin de parcourir les autres. Il garde aussi l'ordre d'ajout, ce qui sert pour l'affichage. J'ai ajouté un deuxième dictionnaire par destination (ville → colis de cette ville).
Temps	O(1) en moyenne pour ajouter ou chercher. En théorie, le pire cas est O(n) si beaucoup d'identifiants tombent au même endroit, mais avec des entiers ça n'arrive pas en pratique.
Espace	O(n). Le dictionnaire garde quelques cases vides pour rester rapide.
Alternative	Une simple liste parcourue du début à la fin : O(n) par recherche. Une liste triée avec une recherche dichotomique : O(log n) pour chercher, mais O(n) pour ajouter, parce qu'il faut décaler les éléments.
Pourquoi ce choix	Pour chercher un colis par son identifiant, on n'a pas besoin d'ordre : le dictionnaire est donc la solution la plus rapide. Mesuré avec 100 000 colis : environ 0,3 µs par recherche avec le dictionnaire, contre environ 4 ms avec une liste, soit environ 10 000 fois plus lent.

Choix 2 : les tris (tri fusion et tri par dénombrement)

Problème	Afficher les colis triés par poids, par valeur ou par priorité.
Solution	**Poids et valeur :** un tri fusion. On découpe la liste en petits morceaux triés, qu'on fusionne deux par deux (blocs de 1, puis 2, 4, 8…) jusqu'à avoir une seule liste triée. Je l'ai écrit sans récursion, pour ne pas atteindre la limite de récursion de Python avec 100 000 colis. **Priorité :** la priorité ne peut valoir que 1, 2, 3, 4 ou 5. Je fais donc 5 « paquets » et je mets chaque colis dans le sien (tri par dénombrement). Il suffit ensuite de mettre les paquets bout à bout. Une fois une liste triée, je la garde en mémoire. Si on redemande le même tri sans avoir ajouté de colis, elle est affichée directement.
Temps	Tri fusion : O(n log n), même dans le pire cas. Dénombrement : O(n), car il n'y a que 5 valeurs. Tri déjà en mémoire : O(1).
Espace	O(n) pour les deux tris, et O(n) pour chaque liste triée gardée en mémoire.
Alternative	Tri par insertion ou tri à bulles : $O(n^2)$, beaucoup trop lent pour 100 000 colis. Tri rapide (quicksort) : rapide en moyenne, mais $O(n^2)$ dans le pire cas. La fonction `sorted()` de Python (Timsort, un mélange de tri fusion et de tri par insertion) a la même complexité que mon tri fusion, mais elle est plus rapide parce qu'elle est écrite en C.
Pourquoi ce choix	Le tri fusion est toujours en O(n log n) et il est « stable » : deux colis de même poids restent dans leur ordre d'arrivée. Pour la priorité, le dénombrement est plus adapté puisqu'il n'y a que 5 valeurs : il est environ 30 fois plus rapide que le tri fusion sur 100 000 colis.

Choix 3 : choisir les colis du camion par programmation dynamique

Problème	Le camion a une capacité maximale W. Il faut choisir des colis dont le poids total ne dépasse pas W et dont la valeur totale est la plus grande possible. C'est le problème classique du « sac à dos ». Essayer toutes les combinaisons est impossible : il y en a 2^n, soit déjà plus de 1000 milliards pour 40 colis.
Solution	J'utilise la programmation dynamique. Je remplis un tableau `meilleur[c]` qui donne la meilleure valeur possible pour une charge de c kg au maximum, pour c allant de 0 à W. Je traite les colis un par un : pour chaque charge c, soit je ne prends pas le colis (la valeur ne change pas), soit je le prends (sa valeur + `meilleur[c - poids]`), et je garde le meilleur des deux. Je note aussi à chaque fois si le colis a été pris. À la fin, je remonte ces notes en partant de c = W pour retrouver la liste des colis à charger. **Amélioration :** avant ce calcul, j'enlève les colis qui ne peuvent pas servir. D'abord ceux qui sont plus lourds que le camion. Ensuite, pour chaque poids, on ne peut pas mettre plus de W/poids colis de ce poids dans le camion. Par exemple, dans un camion de 50 kg, on ne mettra jamais plus de 5 colis de 10 kg. Je garde donc seulement les 5 colis de 10 kg qui ont le plus de valeur. On ne perd rien : si une solution utilisait un autre colis de 10 kg, on pourrait l'échanger avec un des 5 meilleurs sans changer le poids et sans baisser la valeur.
Temps	O(n' × W), où n' est le nombre de colis gardés après le filtrage. Grâce au filtrage, n' ne peut pas dépasser environ W × ln(W), quel que soit le nombre total de colis.
Espace	O(n' × W) pour noter les choix : 1 octet par case, soit environ 4 Mo pour un camion de 1 000 kg.
Alternative	Un algorithme glouton : on trie les colis par valeur/poids et on les met dans l'ordre tant que ça rentre. C'est beaucoup plus rapide, mais pas toujours optimal (voir l'exemple ci-dessous). Autre possibilité : le « branch and bound », exact lui aussi, mais qui peut devenir très lent dans certains cas.
Pourquoi ce choix	L'énoncé demande la **meilleure** valeur possible, pas seulement une combinaison qui rentre dans le camion. Comme les poids sont des nombres entiers de kilos, la programmation dynamique donne toujours la solution optimale. Je l'ai vérifié dans `tests.py` en la comparant à une recherche exhaustive sur 900 petits cas aléatoires. Si le calcul devient trop gros (camion énorme), le programme passe au glouton et indique que le résultat est approché.

Exemple où le glouton se trompe : un camion de 10 kg et trois colis : (6 kg, 7 €), (5 kg, 5 €) et (5 kg, 5 €). Le glouton commence par le colis de 6 kg, qui a le meilleur rapport valeur/poids, puis plus rien ne rentre : il obtient 7 €. Or les deux colis de 5 kg remplissent le camion pour 10 €.

Choix 4 : la règle de priorité

Problème	L'entreprise veut favoriser les colis prioritaires. Je considère que la priorité va de 1 (faible) à 5 (urgent).
Solution	Je garde exactement le même algorithme, mais au lieu de maximiser la valeur, je maximise un score : `score = valeur × (1 + α × (priorité − 1) / 4)` Avec α = 1 (valeur par défaut), un colis de priorité 1 garde sa valeur, un colis de priorité 3 compte pour 1,5 fois sa valeur et un colis de priorité 5 pour 2 fois sa valeur. α se change dans le menu ; avec α = 0, on ignore la priorité. Le poids n'est pas dans la formule, car c'est déjà la contrainte du camion.
Temps / Espace	Les mêmes qu'au choix 3. Calculer le score ne coûte rien de plus.
Alternative	Ajouter un bonus fixe (valeur + 10 × priorité) : un colis à 2 € et un colis à 500 € recevraient le même bonus, ce qui n'a pas trop de sens. Ou charger d'abord tous les colis de priorité 5, puis ceux de priorité 4, etc. : c'est plus strict, mais un petit colis urgent sans valeur pourrait passer devant un colis très cher.
Pourquoi ce choix	La règle est facile à expliquer (« un colis urgent vaut double ») et ne dépend pas de l'unité (euros ou centimes). Le résultat reste optimal pour ce score. Avec α, l'entreprise peut régler l'importance qu'elle donne à la priorité. Sur l'exemple de l'énoncé (A, B, C, D) avec un camion de 13 kg : sans priorité, le programme charge B + C (43 €) ; avec la priorité, il charge A + C, car A est de priorité 5.

Choix 5 : les statistiques mises à jour au fur et à mesure

Problème	Afficher les statistiques (nombre de colis, poids total, valeur totale, répartition par priorité et par ville) sans tout recompter à chaque fois.
Solution	À chaque ajout ou suppression d'un colis, je mets à jour les totaux et les compteurs.
Temps / Espace	O(1) de plus par ajout. L'affichage coûte seulement O(nombre de villes), au lieu de O(n).
Alternative / pourquoi	On pourrait tout recalculer à chaque demande, en O(n). Garder quelques compteurs à jour ne prend presque pas de mémoire et l'affichage est immédiat.

3. Résumé des complexités

Opération	Méthode	Temps	Espace
Ajouter / supprimer un colis	dictionnaire	O(1) en moyenne	O(1)
Chercher par identifiant	dictionnaire	O(1) en moyenne	O(1)
Chercher par ville	dictionnaire par destination	O(k), k = nombre de résultats	O(k)
Afficher les colis	parcours (20 par défaut à l'écran)	O(n)	O(n)
Trier par poids / valeur	tri fusion, résultat gardé en mémoire	O(n log n), puis O(1)	O(n)
Trier par priorité	tri par dénombrement	O(n)	O(n)
Statistiques	compteurs mis à jour	O(nombre de villes)	O(nombre de villes)
Préparer le camion	filtrage + programmation dynamique	O(n log W + n' × W)	O(n' × W)

4. Mesures

Mesures faites avec `python benchmark.py` sur des colis générés aléatoirement (poids de 1 à 30 kg). Les temps changent un peu d'une exécution à l'autre, mais l'ordre de grandeur reste le même.

Nombre de colis	Ajout de tous les colis	Recherche (dictionnaire)	Recherche (liste)	Tri fusion	Tri par priorité	`sorted()`	Mémoire
100	0,1 ms	0,1 µs	2 µs	0,2 ms	< 0,1 ms	< 0,1 ms	0,02 Mo
1 000	0,8 ms	0,1 µs	23 µs	4 ms	0,1 ms	0,2 ms	0,2 Mo
10 000	6 ms	0,1 µs	213 µs	28 ms	1 ms	2 ms	1,9 Mo
100 000	135 ms	0,3 µs	4,2 ms	575 ms	19 ms	32 ms	20 Mo

Nombre de colis	Capacité	Colis gardés après filtrage	Temps (prog. dynamique)	Valeur optimale	Temps (glouton)	Valeur (glouton)
100	100 kg	94	2 ms	884,20 €	0,3 ms	874,08 €
1 000	1 000 kg	988	230 ms	8 384,67 €	3 ms	8 383,64 €
10 000	1 000 kg	3 145	840 ms	16 055,52 €	21 ms	16 055,52 €
100 000	100 kg	389	63 ms	3 391,36 €	50 ms	3 391,36 €
100 000	1 000 kg	3 983	1,2 s	29 254,29 €	89 ms	29 254,29 €

5. Que se passe-t-il quand le nombre de colis augmente ?

Recherche d'un colis

Avec le dictionnaire, le temps de recherche ne bouge presque pas : environ 0,1 µs pour 100 colis et 0,3 µs pour 100 000. Avec une liste, le temps est multiplié par 10 à chaque fois qu'on multiplie le nombre de colis

par 10, ce qui est normal pour du O(n). Le dictionnaire reste donc un bon choix, même pour beaucoup plus de colis. La seule limite est qu'il doit tenir en mémoire.

Tri des colis

Le tri fusion passe de 28 ms pour 10 000 colis à 575 ms pour 100 000. Avec du O(n log n), on s'attendait à environ ×12 ; on mesure un peu plus, sans doute à cause de la mémoire cache du processeur. Ça reste correct pour un menu. Comme le tri est gardé en mémoire, le deuxième affichage est instantané. Pour aller plus vite, on pourrait utiliser `sorted()`, environ 18 fois plus rapide car écrit en C. Pour n'afficher que les 20 premiers colis, on pourrait aussi éviter de trier toute la liste.

Préparation du camion

Sans le filtrage, il faudrait faire 100 000 × 1 000 = 100 millions d'opérations, soit plus d'une minute en Python. Avec le filtrage, il reste au plus environ 4 000 colis, même pour 100 000 colis au départ, et le calcul prend environ 1 seconde. Ce qui fait vraiment augmenter le temps, c'est la **capacité du camion** et non le nombre de colis. Pour un très gros camion (40 tonnes au kilo près), il faudrait arrondir les poids (par tranches de 5 kg par exemple) ou utiliser le glouton.

6. Réponses aux questions

Quel est le choix le plus important ?

Pour moi, c'est la programmation dynamique pour le camion. C'est le seul endroit où un mauvais choix ne rend pas seulement le programme plus lent : il donne un **mauvais résultat**. Avec le glouton, on aurait des chargements qui rentrent dans le camion mais qui ne sont pas forcément les meilleurs. Le filtrage avant le calcul compte aussi beaucoup, parce que c'est lui qui permet de traiter 100 000 colis en environ une seconde.

Qu'est-ce qui poserait problème avec plusieurs millions de colis ?

Surtout la mémoire. En Python, un colis prend environ 200 octets, donc environ 200 Mo par million de colis, plus la mémoire des listes triées gardées. Le tri fusion écrit en Python deviendrait lui aussi lent (plusieurs secondes). Le calcul du camion, lui, tiendrait bien grâce au filtrage. Pour gérer des millions de colis, il faudrait plutôt une base de données (par exemple SQLite, avec un index sur l'identifiant), ou passer en C pour les parties lourdes.

Vitesse ou mémoire ?

J'ai le plus souvent privilégié la vitesse, quand ça ne coûtait pas trop de mémoire :

- je garde les listes triées en mémoire pour ne pas les recalculer ;
- j'ai un deuxième dictionnaire par ville et des compteurs pour les statistiques ;
- pour le camion, je garde tout le tableau des choix. Il existe des méthodes qui utilisent moins de mémoire, mais qui refont une partie des calculs.

J'ai quand même fait attention à la mémoire à certains endroits : `slots=True` sur la classe `Colis` (chaque objet prend moins de place), 1 octet par case dans le tableau des choix, et le filtrage, qui réduit fortement la taille de ce tableau.

Une autre stratégie aurait-elle donné un meilleur chargement ?

Non, pas pour ce problème : la programmation dynamique donne déjà la meilleure valeur possible, et les tests contre la recherche exhaustive le confirment. D'autres méthodes peuvent aller plus vite, mais elles ne peuvent pas faire mieux. Le glouton donne souvent presque la même chose sur des données aléatoires, mais pas toujours (voir l'exemple du choix 3). Pour améliorer le résultat dans la vraie vie, il faudrait plutôt prendre en compte d'autres contraintes : le volume des colis, plusieurs camions ou l'ordre des livraisons.

Et en C ?

Je garderais les mêmes algorithmes : table de hachage, tri fusion, dénombrement pour la priorité, programmation dynamique avec le filtrage. Les complexités ne dépendent pas du langage. Par contre :

- il faudrait coder moi-même la table de hachage et gérer la mémoire avec `malloc` et `free` ;
- les colis seraient dans un tableau de `struct`, beaucoup plus léger (environ 32 octets par colis au lieu de 200) ;
- le programme serait beaucoup plus rapide, donc garder les listes triées en mémoire serait moins utile, et on pourrait gérer des camions plus gros sans arrondir les poids.